give me a simple project which uses simple and imp backend principles it uses microservices based aticheture , have authentication and autherization , used redis and a simple cloud database mondodb small but a project that i can add it to my resume

give me a detailed project explation and everything about that project needed to develpo that project with the help of ai based coding ide

I'll give you a complete, resume-ready microservices project blueprint that hits all your requirements:

- Microservices-based architecture
- Authentication & authorization (JWT + role-based access control)
- Redis for caching and rate limiting
- MongoDB as the cloud database
- Simple but meaningful domain logic
- Designed so you can build it efficiently with an AI coding IDE (e.g., Cursor, Windsurf, Codeium, etc.)

Project Idea: "TaskFlow" – A Microservices Task & Team Management API

One-line pitch:
A backend-only microservices system for managing teams, users, and tasks, with secure auth, role-based permissions, caching, and MongoDB persistence—exposed via REST APIs.

This is small enough to finish in 1–2 weeks part-time, but complex enough to show serious backend skills on your resume.

Why This Is Good for Your Resume

- Demonstrates **microservices architecture** (multiple services, inter-service communication).
- Shows **authN/authZ** done properly (JWT, roles, protected routes).
- Uses **Redis** for:
  - Caching frequently accessed data (e.g., user profile, team info)

    - Rate limiting API endpoints
- Uses **MongoDB** (cloud-hosted, e.g., MongoDB Atlas) as the primary DB.
- Clear separation of concerns, scalable design, and modern backend patterns.
- You can deploy it on a free/low-cost cloud (Render, Railway, Fly.io, etc.) and link to a live API + Postman collection / OpenAPI spec.

High-Level Architecture

Services

1. **Auth Service**
    - Handles user registration, login, token issuance.
    - Stores users in MongoDB.
    - Issues JWT access tokens (and optionally refresh tokens).
    - Exposes:
        - `POST /auth/register`
        - `POST /auth/login`
        - `POST /auth/refresh`
        - `GET /auth/me` (validate token, return user info)
2. **User Service**
    - Manages user profiles and basic user data.
    - Reads/writes user documents in MongoDB.
    - Caches user profiles in Redis.
    - Exposes:
        - `GET /users/:id`
        - `PUT /users/:id`
        - `GET /users/me`
3. **Team Service**
    - Manages teams and memberships.
    - Each team has:
        - Name, description
        - Members (user IDs + roles: `OWNER`, `ADMIN`, `MEMBER`)
    - Caches team data in Redis.
    - Exposes:
        - `POST /teams`
        - `GET /teams/:id`

- `PUT /teams/:id`
- `DELETE /teams/:id`
- `POST /teams/:id/members`
- `DELETE /teams/:id/members/:userId`
- `GET /teams/:id/members`

4. **Task Service**
   - Manages tasks within teams.
   - Each task has:
     - Title, description, status (`TODO`, `IN_PROGRESS`, `DONE`)
     - Assignee (user ID)
     - Team ID
     - CreatedBy, timestamps
   - Uses Redis to cache "task lists by team" for faster reads.
   - Exposes:
     - `POST /tasks`
     - `GET /tasks/:id`
     - `PUT /tasks/:id`
     - `DELETE /tasks/:id`
     - `GET /teams/:id/tasks`

Shared Infrastructure

- **API Gateway (optional but nice)**
  A thin Node.js/Go/FastAPI service that:
  - Routes `/auth/*` → Auth Service
  - Routes `/users/*` → User Service
  - Routes `/teams/*` → Team Service
  - Routes `/tasks/*` → Task Service
    You can also skip a separate gateway initially and just run services on different ports/paths and document them.
- **Redis**
  - Used by:
    - Auth Service: store refresh tokens / blacklisted tokens (optional).
    - User Service: cache user profiles (`user:{id}`).
    - Team Service: cache team details (`team:{id}`).
    - Task Service: cache team task lists (`team:{id}:tasks`).
  - Also used for simple **rate limiting** (e.g., per IP or per user ID).

- **MongoDB (cloud)**
  - Collections:
    - `users`
    - `teams`
    - `tasks`
  - Hosted on MongoDB Atlas (free tier).
- **Inter-service communication**
  - Synchronous HTTP (REST) calls between services (simple and fine for a small project).
  - Example: Task Service calls User Service to verify assignee exists; Team Service calls Auth Service to validate user.

Core Backend Principles You'll Demonstrate

You'll explicitly implement and be able to talk about these in interviews:

1. **Separation of Concerns**
   - Each service has its own domain: auth, users, teams, tasks.
   - Clear layers: routes → controllers → services → repositories.
2. **Stateless Services + Centralized State**
   - Services don't store session state locally.
   - Auth via JWT; state in MongoDB + Redis.
3. **Authentication & Authorization**
   - JWT-based auth.
   - Middleware to:
     - Verify JWT.
     - Extract user ID & roles.
     - Enforce role-based access (e.g., only team `OWNER`/`ADMIN` can delete team).
4. **Caching Strategy**
   - Cache-aside pattern:
     - On read: check Redis → if miss, read from MongoDB → write to Redis.
     - On write: update DB + invalidate/update cache.
   - You can explain TTLs, cache invalidation, and trade-offs.
5. **Rate Limiting**
   - Use Redis counters to limit requests per minute per IP/user.
   - Middleware that returns `429 Too Many Requests` when limit exceeded.
6. **Error Handling & Logging**

    - Centralized error-handling middleware.
    - Structured logs (JSON) with request ID, user ID, service name.
7. **Config Management**
    - Environment variables for:
        - DB URI
        - Redis URI
        - JWT secret
        - Service URLs
    - `.env` files locally, secrets in cloud environment.
8. **API Design**
    - RESTful routes, proper HTTP verbs and status codes.
    - Pagination for list endpoints (e.g., `GET /teams/:id/tasks?page=1&limit=20`).

Tech Stack (Simple & Popular)

Pick one primary language; Node.js + TypeScript is very AI-IDE-friendly:

- **Language**: TypeScript (Node.js)
- **Framework**: Fastify or Express
- **DB Driver**: `mongodb` or `mongoose`
- **Redis Client**: `ioredis` or `redis`
- **JWT**: `jsonwebtoken`
- **Password hashing**: `bcrypt` or `argon2`
- **Validation**: `zod` or `joi`
- **Logging**: `pino` or `winston`
- **Testing**: `vitest` or `jest` + `supertest`
- **Containerization**: Docker + `docker-compose` for local dev
- **Deployment**: Render/Railway/Fly.io (each service as a separate app) or a single repo with multiple services deployed separately.

If you prefer Python, you can do the same with FastAPI + `motor` (async MongoDB) + `redis-py`.

Data Models (MongoDB Schemas)

Users (`users` collection)

```
{
  "_id": "ObjectId",
  "email": "ravi@example.com",
  "passwordHash": "...",
  "name": "Ravi Sharma",
  "role": "USER",
  "createdAt": "ISODate",
  "updatedAt": "ISODate"
}
```

- `role` can be `USER` or `ADMIN` (for platform-level admin).

Teams (`teams` collection)

```
{
  "_id": "ObjectId",
  "name": "Backend Team",
  "description": "Team for backend services",
  "members": [
    {
      "userId": "ObjectId",
      "role": "OWNER" // or "ADMIN", "MEMBER"
    }
  ],
  "createdAt": "ISODate",
  "updatedAt": "ISODate"
}
```

Tasks (`tasks` collection)

```
{
  "_id": "ObjectId",
  "teamId": "ObjectId",
  "title": "Implement auth middleware",
  "description": "Add JWT verification...",
  "status": "TODO", // "TODO" | "IN_PROGRESS" | "DONE"
  "assigneeId": "ObjectId",
  "createdBy": "ObjectId",
  "createdAt": "ISODate",
  "updatedAt": "ISODate"
}
```

Add indexes on:

- `users.email` (unique)
- `teams.members.userId`
- `tasks.teamId`, `tasks.assigneeId`

Authentication & Authorization Flow

Registration & Login

1. `POST /auth/register`
   - Body: `{ email, password, name }`
   - Validate input.
   - Check if email exists in MongoDB.
   - Hash password.
   - Save user.
   - Return `{ user, accessToken }`.
2. `POST /auth/login`
   - Body: `{ email, password }`
   - Find user by email.
   - Verify password.
   - Generate JWT:
     - Payload: `{ userId, email, role }`
     - Short expiry (e.g., 15 min).
   - Return `{ user, accessToken }`.

JWT Middleware

- Applied to all protected routes (`/users/*`, `/teams/*`, `/tasks/*`).
- Steps:
  1. Extract `Authorization: Bearer <token>`.
  2. Verify JWT with secret.
  3. Attach `user` object to request context: `req.user = { userId, email, role }`.
  4. If invalid/expired → `401 Unauthorized`.

Role-Based Authorization

- Example: Only team `OWNER` or `ADMIN` can delete a team.
- In Team Service:
  - Get `teamId` from route.
  - Load team (from cache or DB).
  - Find member entry for `req.user.userId`.
  - If role not in allowed set → `403 Forbidden`.

Same pattern for tasks:

- Only assignee, creator, or team admin can update/delete a task.

Redis Usage Details

Caching

User profile cache:

- Key: `user:{userId}`
- Value: JSON string of user (without passwordHash).
- TTL: e.g., 10 minutes.
- Flow:
  - `GET /users/:id`:
    - Check Redis.
    - If present → return.
    - Else → query MongoDB → set cache → return.

Team cache:

- Key: `team:{teamId}`
- Similar pattern.

Task list cache:

- Key: `team:{teamId}:tasks`
- Value: array of tasks (or summarized version).
- Invalidate on task create/update/delete for that team.

Rate Limiting

- Key: `ratelimit:{userId or ip}`
- Value: counter
- TTL: 1 minute
- Middleware:
  - Increment counter.
  - If > N requests in window → return `429`.

You can implement a simple fixed-window rate limiter using Redis `INCR` + `EXPIRE`.

Project Structure (Monorepo Example)

You can keep all services in one repo for simplicity:

```
taskflow/
  apps/
    auth-service/
      src/
        routes/
        controllers/
        services/
        repositories/
        middleware/
        utils/
        index.ts
      Dockerfile
      package.json
    user-service/
      ...
    team-service/
      ...
    task-service/
      ...
  shared/
    types/
    utils/
  docker-compose.yml
  README.md
  docs/
    api.md
    architecture.md
```

Each service:

- Has its own `package.json`, `Dockerfile`, and entry point.
- Shares common types via `shared/`.

How to Develop This With an AI Coding IDE

Assume you're using something like Cursor, Windsurf, or Codeium.

Step 1: Set Up the Repo & Infra

1. Create a new repo: `taskflow-backend`.
2. In your AI IDE, prompt:
   > "Create a Node.js + TypeScript monorepo with four services: auth-service, user-service, team-service, task-service. Each service should be an Express app with a basic health endpoint `/health`. Use pnpm workspaces or npm workspaces. Add a docker-compose.yml that runs MongoDB and Redis locally."
3. Let the AI generate:

- Folder structure
- Basic `package.json` files
- `tsconfig.json`
- `docker-compose.yml` with:
  - `mongo` service (mongo:7)
  - `redis` service (redis:7)
  - Optional: one service container for testing.

4. Run locally:

```
docker-compose up -d
```

Step 2: Implement Auth Service

Prompt your AI IDE:

> "In `apps/auth-service`, implement:
>
> - MongoDB connection using mongoose (or mongodb driver).
> - User model with email, passwordHash, name, role.
> - Routes: POST /auth/register, POST /auth/login, GET /auth/me.
> - Use bcrypt for password hashing.
> - Use jsonwebtoken for JWT.
> - Add middleware to verify JWT for /auth/me.
> - Use environment variables for MONGODB_URI, JWT_SECRET, PORT.
> - Add basic input validation with zod."

Then:

- Ask it to add:
  - Error-handling middleware.
  - Logging with pino.
  - Simple rate limiting using Redis (you'll wire Redis later).

Once generated:

- Test with Postman/curl:
  - Register a user.
  - Login and get a token.
  - Call `/auth/me` with `Authorization: Bearer <token>`.

Step 3: Implement User Service

Prompt:

> "In `apps/user-service`, implement:
>
> - MongoDB connection to the same DB.
> - User repository (read/update user by ID).
> - Redis client connection.
> - Caching: on GET /users/:id, check Redis key `user:{id}`, else read from DB and cache.
> - Invalidate cache on PUT /users/:id.
> - Add JWT middleware that calls auth-service to validate token or verifies JWT directly using shared secret.
> - Expose GET /users/:id, PUT /users/:id, GET /users/me."

You can choose:

- Either share JWT secret and verify locally (simpler).
- Or call Auth Service's `/auth/verify` endpoint (more microservice-y).

Step 4: Implement Team Service

Prompt:

> "In `apps/team-service`, implement:
>
> - Team model with name, description, members array (userId + role).
> - Endpoints:
>   - POST /teams (create team, caller becomes OWNER)
>   - GET /teams/:id
>   - PUT /teams/:id (only OWNER/ADMIN)
>   - DELETE /teams/:id (only OWNER)
>   - POST /teams/:id/members (add member, only OWNER/ADMIN)
>   - DELETE /teams/:id/members/:userId (only OWNER)
>   - GET /teams/:id/members
> - Use Redis to cache team by `team:{id}`.
> - Invalidate cache on updates.
> - Add authorization checks based on team member roles.
> - Use JWT middleware for auth."

Step 5: Implement Task Service

Prompt:

> "In `apps/task-service`, implement:
>
> - Task model with title, description, status, assigneeId, teamId, createdBy.
> - Endpoints:
>   - POST /tasks (create task in a team; caller must be member of that team)
>   - GET /tasks/:id
>   - PUT /tasks/:id (assignee, creator, or team admin can update)
>   - DELETE /tasks/:id (same rules)
>   - GET /teams/:id/tasks (list tasks for a team, with pagination)
> - Use Redis to cache `team:{teamId}:tasks` list.
> - Invalidate cache on task create/update/delete.
> - Add authorization checks:
>   - Only team members can access tasks in that team.
> - Use JWT middleware for auth."

Step 6: Add Global Rate Limiting

In each service (or in a shared middleware package):

Prompt:

> "Add a rate-limiting middleware using Redis:
>
> - Key: `ratelimit:{userId or ip}`
> - Window: 1 minute
> - Max requests: 60
> - Return 429 with JSON `{ error: 'Too many requests' }` when exceeded.
> - Apply to all routes except `/health`."

You can derive user ID from JWT if present, else use IP.

Step 7: Dockerize & Deploy

1. Add `Dockerfile` for each service:
   - Node base image.
   - Copy code, install deps, build TS, run.
2. Update `docker-compose.yml` to include all 4 services + mongo + redis for local testing.
3. For deployment:
   - Create a MongoDB Atlas cluster (free tier).

- Create a Redis Cloud free instance (or use Redis on Render/Railway).
- Deploy each service as a separate app (or all in one repo with different start commands if your platform supports it).
- Set env vars:
  - `MONGODB_URI`
  - `REDIS_URI`
  - `JWT_SECRET`
  - `AUTH_SERVICE_URL`, `USER_SERVICE_URL`, etc. (if services call each other).

What to Put on Your Resume

Under **Projects**:

TaskFlow – Microservices Task & Team Management Backend

- Designed and implemented a microservices-based REST API with 4 services (auth, user, team, task) using Node.js, TypeScript, and Express.
- Implemented JWT-based authentication and role-based authorization (OWNER/ADMIN/MEMBER) for teams and tasks.
- Integrated Redis for caching user/team/task data and for per-user rate limiting, reducing average read latency by ~X%.
- Used MongoDB Atlas as the cloud database with proper indexing and schema design.
- Containerized services with Docker and deployed on [Render/Railway], exposing documented APIs with OpenAPI/Postman collection.
- Applied backend best practices: separation of concerns, stateless services, centralized error handling, structured logging, and environment-based configuration.

Add:

- GitHub repo link.
- Live API base URL + Postman collection or OpenAPI JSON.
- Optional: a small diagram (you can draw in Excalidraw) showing services, Redis, MongoDB.

How to Use AI IDE Effectively for This Project

- **Generate boilerplate fast**: Let AI create folder structures, basic routes, models, and middleware.
- **Iterate on logic**: Ask for specific features ("add pagination to GET /teams/:id/tasks").
- **Refactor**: "Refactor this controller to use a service layer and repository pattern."
- **Tests**: "Generate unit tests for the auth service login flow using vitest and supertest."

- **Docs**: “Generate an OpenAPI spec for all endpoints in this service.”
- **Debugging**: Paste error logs and ask AI to suggest fixes.

But always:

- Review generated code.
- Ensure you understand auth flows, caching, and authorization logic so you can explain them in interviews.

Optional Enhancements (If You Have Time)

- Add **refresh tokens** stored in Redis.
- Add **audit logs** collection for important actions (team created, task deleted).
- Add **simple analytics** endpoint: “tasks per status per team” (shows aggregation skills).
- Add **OpenAPI/Swagger UI** for interactive docs.
- Add **CI** (GitHub Actions) to run tests and lint on push.

If you tell me your preferred language (Node/TS vs Python/FastAPI vs something else), I can outline exact commands and file templates to get you started in your AI IDE.